

Neck: 多尺度特征融合 (FPN)

为什么需要多尺度?

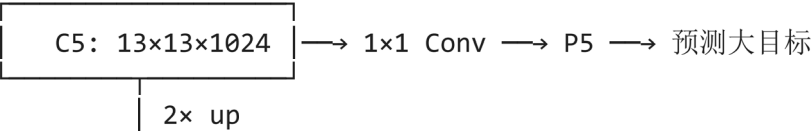
YOLOv3 的 Darknet-53 backbone 输出了 **3 个尺度的特征图**:

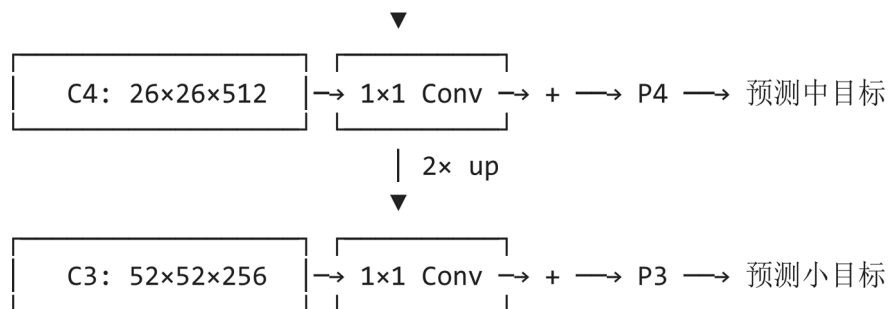
特征图	下采样倍数	感受野	擅长检测
52 × 52	8×	小	小目标 (鸟、猫)
26 × 26	16×	中	中目标 (狗、人)
13 × 13	32×	大	大目标 (车、牛)

问题: 浅层特征 (52×52) 分辨率高但语义弱, 深层特征 (13×13) 语义强但分辨率低。

FPN 的解决方案: 把深层语义信息**自顶向下**传递到浅层, 让每一层既有高分辨率又有强语义。

FPN 核心结构

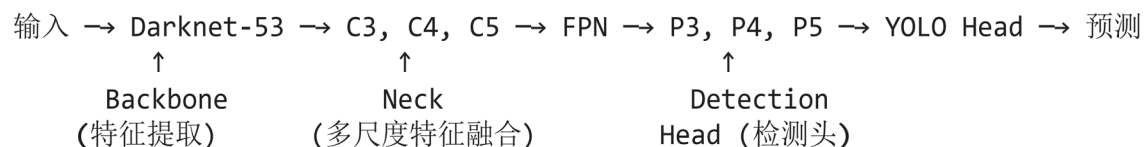




三个关键操作

1. **1×1 卷积 (Lateral Connection)**：将各层通道数统一到相同维度 (如 256)，便于融合
2. **$2 \times$ 上采样 (Top-down)**：将高层特征图放大到与下一层相同尺寸
3. **逐元素相加**：融合深层语义 + 浅层细节

与 YOLOv3 的关系



FPN 连接了 Backbone 和 Detection Head，因此被称为 **Neck (颈部)**。

```
In [2]: import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
import numpy as np

# =====
# 修复中文字体显示
# =====
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Arial Unicode MS']
```

```
plt.rcParams['axes.unicode_minus'] = False

# =====
# 1. 基础组件: Conv + BN + LeakyReLU
# =====
class ConvBlock(nn.Module):
    """标准卷积块: Conv2d + BatchNorm + LeakyReLU"""
    def __init__(self, in_ch, out_ch, kernel_size=1, stride=1):
        super().__init__()
        padding = kernel_size // 2
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size, stride, padding, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.LeakyReLU(0.1, inplace=True),
        )
    def forward(self, x):
        return self.conv(x)

# =====
# 2. FPN 核心实现
# =====
class FPN(nn.Module):
    """
    Feature Pyramid Network (FPN)

    输入: Darknet-53 backbone 输出的三个尺度特征图
    - C3: 52×52×256 (下采样8倍)
    - C4: 26×26×512 (下采样16倍)
    - C5: 13×13×1024 (下采样32倍)

    输出: 融合后的三个尺度特征图
    - P3: 52×52×256 (语义增强的小目标检测)
    - P4: 26×26×256 (中目标检测)
    - P5: 13×13×256 (大目标检测)

    结构:
    C5 → 1×1 Conv → P5 → 预测大目标
        ↑           |
        Lateral    2× up
    """
```

```

"""
def __init__(self,
               in_channels_list=[256, 512, 1024], # C3, C4, C5 的通道数
               out_channels=256):                 # 统一输出通道数
    super().__init__()

    # ---- 1x1 Lateral Connections (侧连接: 调整通道数) ----
    self.lateral_convs = nn.ModuleList([
        nn.Conv2d(in_ch, out_channels, kernel_size=1)
        for in_ch in in_channels_list
    ])

    # ---- 3x3 Conv (平滑融合后的特征, 减少上采样锯齿) ----
    self.smooth_convs = nn.ModuleList([
        ConvBlock(out_channels, out_channels, kernel_size=3)
        for _ in range(3)
    ])

def forward(self, features):
    """
    Args:
        features: tuple/list of 3 feature maps [C3, C4, C5]
                  形状: [(B,256,52,52), (B,512,26,26), (B,1024,13,13)]

    Returns:
        [P3, P4, P5]: 融合后的特征图, 都是 (B, 256, H, W)
    """
    c3, c4, c5 = features

    # ---- Top-down pathway (自顶向下融合) ----
    # Step 1: 对最深层 C5 做 1x1 卷积, 得到 P5
    p5 = self.lateral_convs[2](c5) # (B, 256, 13, 13)

    # Step 2: P5 上采样 → 与 C4 的 1x1 卷积结果相加 → P4
    p5_up = F.interpolate(p5, size=c4.shape[2:], mode='nearest') # 2x up
    p4 = self.lateral_convs[1](c4) + p5_up # (B, 256, 26, 26)

```

```

# Step 3: P4 上采样 → 与 C3 的 1x1 卷积结果相加 → P3
p4_up = F.interpolate(p4, size=c3.shape[2:], mode='nearest') # 2x up
p3 = self.lateral_convs[0](c3) + p4_up # (B, 256, 52, 52)

# ---- 3x3 平滑卷积 (消除上采样的重叠效应) ----
p3 = self.smooth_convs[0](p3)
p4 = self.smooth_convs[1](p4)
p5 = self.smooth_convs[2](p5)

return [p3, p4, p5]

# =====
# 3. 模拟 Backbone + FPN + Head 的完整流程
# =====
class TinyBackbone(nn.Module):
    """简化的模拟 backbone, 输出3个尺度的特征图 (模拟 Darknet-53)"""
    def __init__(self):
        super().__init__()
        # 用简单卷积模拟不同尺度的特征提取
        self.stem = nn.Sequential(
            ConvBlock(3, 64, 3, stride=2), # 208x208
            ConvBlock(64, 128, 3, stride=2), # 104x104
        )
        # C3: 52x52x256 (下采样8倍)
        self.c3_conv = nn.Sequential(
            ConvBlock(128, 256, 3, stride=2),
            ConvBlock(256, 256, 3),
        )
        # C4: 26x26x512 (下采样16倍)
        self.c4_conv = nn.Sequential(
            ConvBlock(256, 512, 3, stride=2),
            ConvBlock(512, 512, 3),
        )
        # C5: 13x13x1024 (下采样32倍)
        self.c5_conv = nn.Sequential(
            ConvBlock(512, 1024, 3, stride=2),
            ConvBlock(1024, 1024, 3),
        )

```

```

def forward(self, x):
    x = self.stem(x)
    c3 = self.c3_conv(x)
    c4 = self.c4_conv(c3)
    c5 = self.c5_conv(c4)
    return [c3, c4, c5]

class YOLOHead(nn.Module):
    """简化的 YOLO 检测头：对每个尺度做预测"""
    def __init__(self, in_channels=256, num_classes=6):
        super().__init__()
        self.pred = nn.Conv2d(in_channels, 5 + num_classes, kernel_size=1)

    def forward(self, x):
        return self.pred(x)

# =====
# 4. 实例化并验证维度
# =====
print("=" * 70)
print("    FPN 多尺度特征融合 - 维度验证")
print("=" * 70)

# 构建完整模型
backbone = TinyBackbone()
fpn = FPN(in_channels_list=[256, 512, 1024], out_channels=256)
heads = nn.ModuleList([YOLOHead(256, num_classes=6) for _ in range(3)])

# 模拟输入
x = torch.randn(1, 3, 416, 416)

# 前向传播
with torch.no_grad():
    # Step 1: Backbone 提取特征
    c3, c4, c5 = backbone(x)
    print(f"\n📦 Backbone 输出 (3个尺度):")
    print(f"    C3: {tuple(c3.shape)} (下采样8倍, 浅层高分辨率)")
    print(f"    C4: {tuple(c4.shape)} (下采样16倍, 中层)")
    print(f"    C5: {tuple(c5.shape)} (下采样32倍, 深层强语义)")

```

```

# Step 2: FPN 多尺度融合
p3, p4, p5 = fpn([c3, c4, c5])
print(f"\n🔗 FPN 融合后 (通道统一为 256):")
print(f"    P3: {tuple(p3.shape)} ← 融合了 C5→C4→C3 的语义信息")
print(f"    P4: {tuple(p4.shape)} ← 融合了 C5→C4 的语义信息")
print(f"    P5: {tuple(p5.shape)} ← 最深层, 语义最强")

# Step 3: YOLO Head 预测
preds = [head(p) for head, p in zip(heads, [p3, p4, p5])]
print(f"\n🔗 YOLO Head 预测输出 (每个尺度11通道: 4+1+6):")
for i, pred in enumerate(preds):
    scale_name = ['小目标', '中目标', '大目标'][i]
    print(f"    尺度{i+1} ({scale_name}): {tuple(pred.shape)}")

print(f"\n✅ 总参数量: {sum(p.numel() for p in fpn.parameters())/1e3:.1f}K")

# =====
# 5. 可视化: FPN 融合前后的特征图对比
# =====
def visualize_fusion(c_before, p_after, stage_name):
    """对比融合前后的特征图"""
    fig, axes = plt.subplots(2, 4, figsize=(16, 6))

    c_np = c_before[0, :4].cpu().numpy()
    p_np = p_after[0, :4].cpu().numpy()

    for i in range(4):
        # 融合前
        ax1 = axes[0, i]
        im1 = ax1.imshow(c_np[i], cmap='viridis')
        ax1.set_title(f'融合前 Ch{i}', fontsize=9)
        ax1.axis('off')
        plt.colorbar(im1, ax=ax1, fraction=0.046)

        # 融合后
        ax2 = axes[1, i]
        im2 = ax2.imshow(p_np[i], cmap='plasma')
        ax2.set_title(f'融合后 Ch{i}', fontsize=9)
        ax2.axis('off')

```

```

plt.colorbar(im2, ax=ax2, fraction=0.046)

plt.suptitle(f'{stage_name} - FPN 融合前后特征对比 (前4通道)', fontsize=13)
plt.tight_layout()
plt.show()

with torch.no_grad():
    visualize_fusion(c3, p3, '尺度1 (52×52) - 小目标检测')
    visualize_fusion(c4, p4, '尺度2 (26×26) - 中目标检测')
    visualize_fusion(c5, p5, '尺度3 (13×13) - 大目标检测')

# =====
# 6. 结构可视化: FPN 信息流图 (纯文本 + matplotlib)
# =====
print("\n" + "=" * 70)
print("      FPN 结构图")
print("=" * 70)

fig, ax = plt.subplots(1, 1, figsize=(12, 8))
ax.set_xlim(0, 12)
ax.set_ylim(0, 10)
ax.axis('off')

def draw_box(ax, x, y, w, h, text, color='#4ecdc4', text_color='white', fontsize=10):
    rect = plt.Rectangle((x, y), w, h, facecolor=color, edgecolor='#333',
                          linewidth=2, alpha=0.85)
    ax.add_patch(rect)
    ax.text(x + w/2, y + h/2, text, ha='center', va='center',
            color=text_color, fontsize=fontsize, fontweight='bold')

def draw_arrow(ax, x1, y1, x2, y2, color='#666', lw=2):
    ax.annotate('', xy=(x2, y2), xytext=(x1, y1),
                arrowprops=dict(arrowstyle='->', color=color, lw=lw,
                                connectionstyle='arc3,rad=0'))

# Backbone 输出
draw_box(ax, 0.5, 7, 2.5, 1.2, 'Backbone\nDarknet-53', '#6c5ce7')
draw_box(ax, 1, 5.5, 2, 0.8, 'C3: 52²×256', '#a29bfe', fontsize=9)
draw_box(ax, 1, 4, 2, 0.8, 'C4: 26²×512', '#a29bfe', fontsize=9)
draw_box(ax, 1, 2.5, 2, 0.8, 'C5: 13²×1024', '#a29bfe', fontsize=9)

```



```
# Lateral 1x1 Conv
draw_box(ax, 4, 5.3, 1.5, 0.7, '1x1\nConv', '#fdb66e', fontsize=8)
draw_box(ax, 4, 3.8, 1.5, 0.7, '1x1\nConv', '#fdb66e', fontsize=8)
draw_box(ax, 4, 2.3, 1.5, 0.7, '1x1\nConv', '#fdb66e', fontsize=8)

# 箭头: C → Lateral
draw_arrow(ax, 3, 5.9, 4, 5.65)
draw_arrow(ax, 3, 4.4, 4, 4.15)
draw_arrow(ax, 3, 2.9, 4, 2.65)

# Top-down 路径
draw_box(ax, 6.5, 2.3, 1.2, 0.7, 'P5\n13²', '#00b894', fontsize=9)
draw_arrow(ax, 5.5, 2.65, 6.5, 2.65)

# 上采样 P5 → P4
draw_box(ax, 7.5, 3.5, 0.8, 0.5, '2× up', '#e17055', fontsize=7)
draw_arrow(ax, 7.1, 2.8, 7.5, 3.5)
draw_arrow(ax, 5.5, 4.15, 7.5, 3.5)

# P4 相加节点
draw_box(ax, 8.5, 3.2, 0.6, 0.6, '+', '#e17055', fontsize=12)
draw_arrow(ax, 8.3, 3.5, 8.5, 3.5)

draw_box(ax, 9.5, 3.5, 1.2, 0.7, 'P4\n26²', '#00b894', fontsize=9)
draw_arrow(ax, 9.1, 3.5, 9.5, 3.5)

# 上采样 P4 → P3
draw_box(ax, 7.5, 5, 0.8, 0.5, '2× up', '#e17055', fontsize=7)
draw_arrow(ax, 10.7, 3.8, 10.7, 5.5)
draw_arrow(ax, 5.5, 5.65, 7.5, 5)

# P3 相加节点
draw_box(ax, 8.5, 4.7, 0.6, 0.6, '+', '#e17055', fontsize=12)
draw_box(ax, 9.5, 5, 1.2, 0.7, 'P3\n52²', '#00b894', fontsize=9)
draw_arrow(ax, 9.1, 5, 9.5, 5)

# 预测头
draw_box(ax, 11, 5, 1, 0.6, 'Head\n小目标', '#0984e3', fontsize=8)
draw_box(ax, 11, 3.5, 1, 0.6, 'Head\n中目标', '#0984e3', fontsize=8)
draw_box(ax, 11, 2, 1, 0.6, 'Head\n大目标', '#0984e3', fontsize=8)
```

```

draw_arrow(ax, 10.7, 5.3, 11, 5.3)
draw_arrow(ax, 10.7, 3.8, 11, 3.8)
draw_arrow(ax, 10.7, 2.3, 11, 2.3)

ax.set_title('FPN (Feature Pyramid Network) - 完整信息流', fontsize=14, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

# =====
# 7. FPN 融合细节: 验证上采样 + 相加的效果
# =====
print("\n" + "=" * 70)
print("    FPN 融合细节验证")
print("=" * 70)

with torch.no_grad():
    # 单独看 P5 → P4 的融合过程
    p5_before_smooth = fpn.lateral_convs[2](c5) # (B, 256, 13, 13)
    p5_up = F.interpolate(p5_before_smooth, size=c4.shape[2:], mode='nearest')
    c4_lat = fpn.lateral_convs[1](c4)
    p4_before_smooth = c4_lat + p5_up

    print(f"\n📁 P4 融合过程维度变化:")
    print(f"    C5 lateral → P5:      {tuple(p5_before_smooth.shape)}")
    print(f"    P5 上采样 2×:         {tuple(p5_up.shape)}")
    print(f"    C4 lateral (1×1 Conv): {tuple(c4_lat.shape)}")
    print(f"    P4 = C4_lat + P5_up:   {tuple(p4_before_smooth.shape)} ✅ 维度一致")
    print(f"    P4 平滑后:            {tuple(p4.shape)}")

print(f"\n📁 FPN 参数量明细:")
lat_params = sum(p.weight.numel() + (p.bias.numel() if p.bias is not None else 0) for p in fpn.lateral_convs)
smooth_params = sum(p.conv[0].weight.numel() + (p.conv[0].bias.numel() if p.conv[0].bias is not None else 0) for p in fpn.smooth_convs)
print(f"    Lateral 1×1 Conv: {lat_params/1e3:.1f}K")
print(f"    Smooth 3×3 Conv:  {smooth_params/1e3:.1f}K")
print(f"    FPN 总参数量:      {(lat_params+smooth_params)/1e3:.1f}K")
print(f"    (仅为 Darknet-53 的 0.1%, 非常轻量!)")

```

=====

FPN 多尺度特征融合 - 维度验证

=====



Backbone 输出 (3个尺度):

C3: (1, 256, 52, 52) (下采样8倍, 浅层高分辨率)

C4: (1, 512, 26, 26) (下采样16倍, 中层)

C5: (1, 1024, 13, 13) (下采样32倍, 深层强语义)



FPN 融合后 (通道统一为 256):

P3: (1, 256, 52, 52) ← 融合了 C5→C4→C3 的语义信息

P4: (1, 256, 26, 26) ← 融合了 C5→C4 的语义信息

P5: (1, 256, 13, 13) ← 最深层, 语义最强



YOLO Head 预测输出 (每个尺度11通道: 4+1+6):

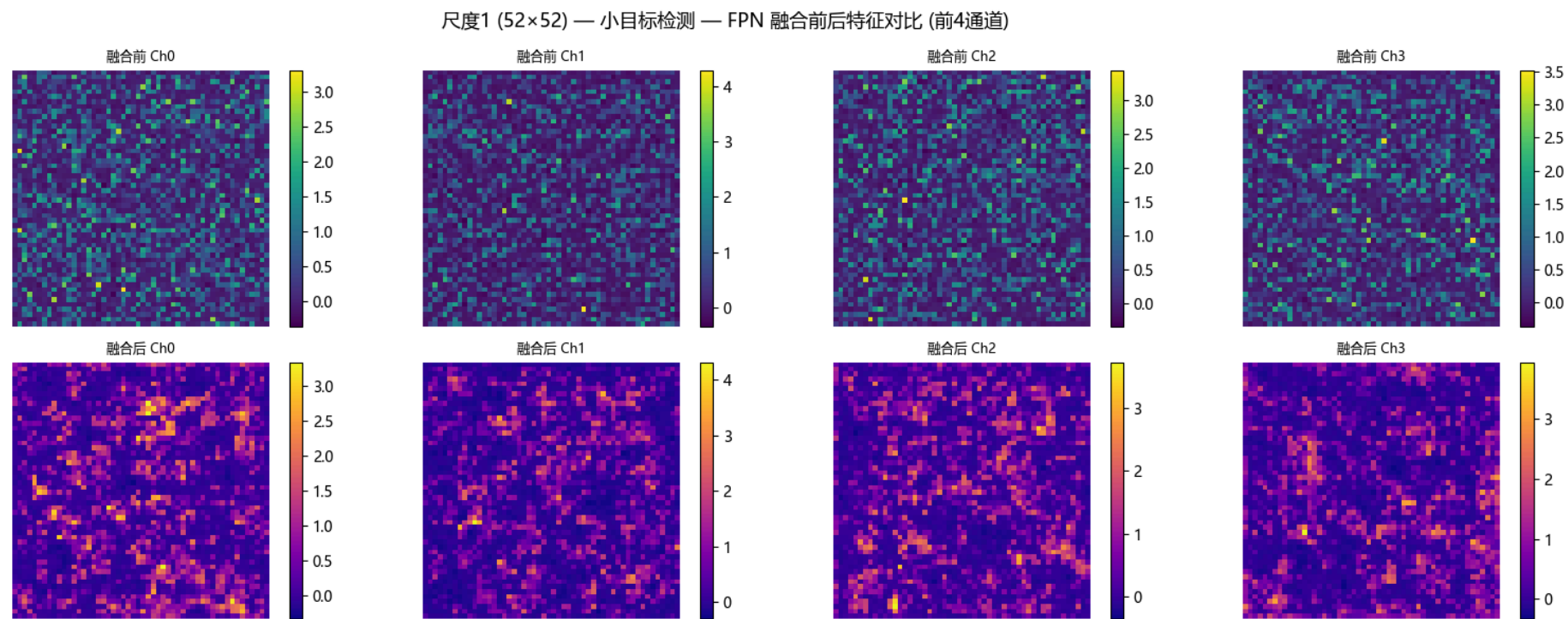
尺度1 (小目标): (1, 11, 52, 52)

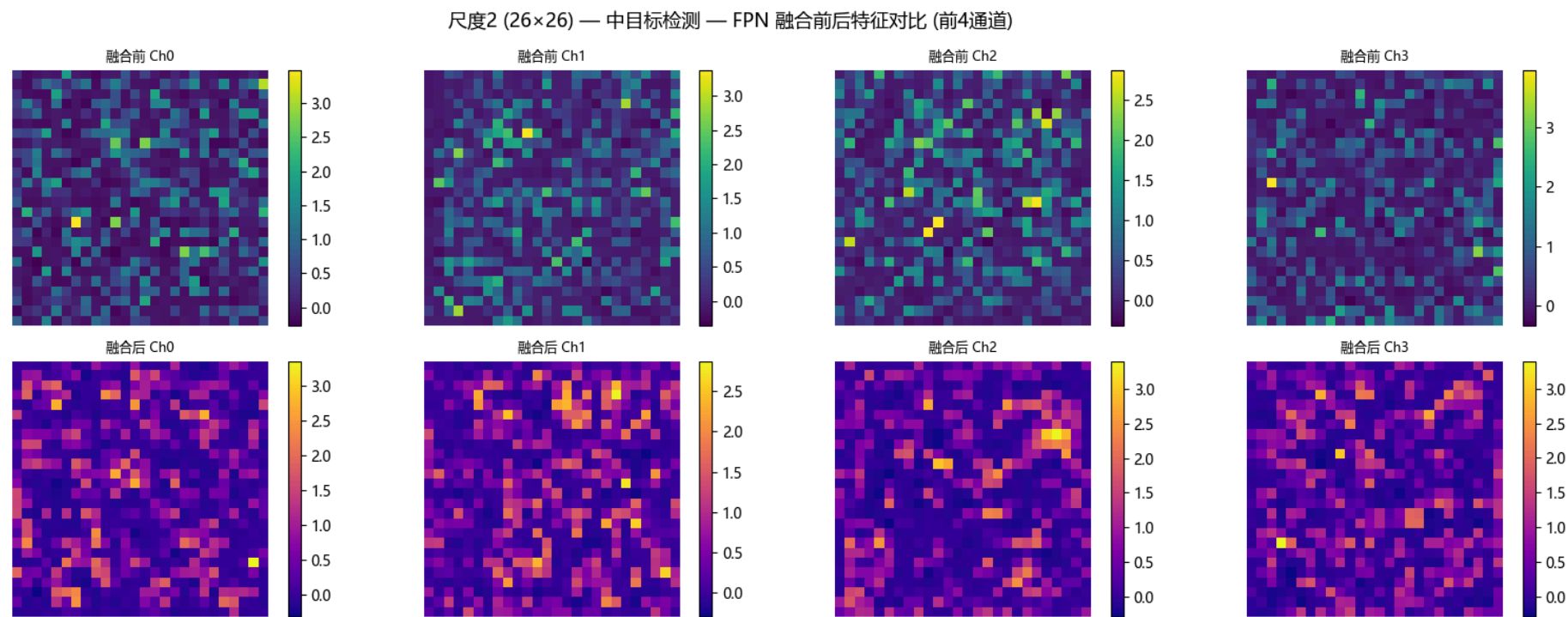
尺度2 (中目标): (1, 11, 26, 26)

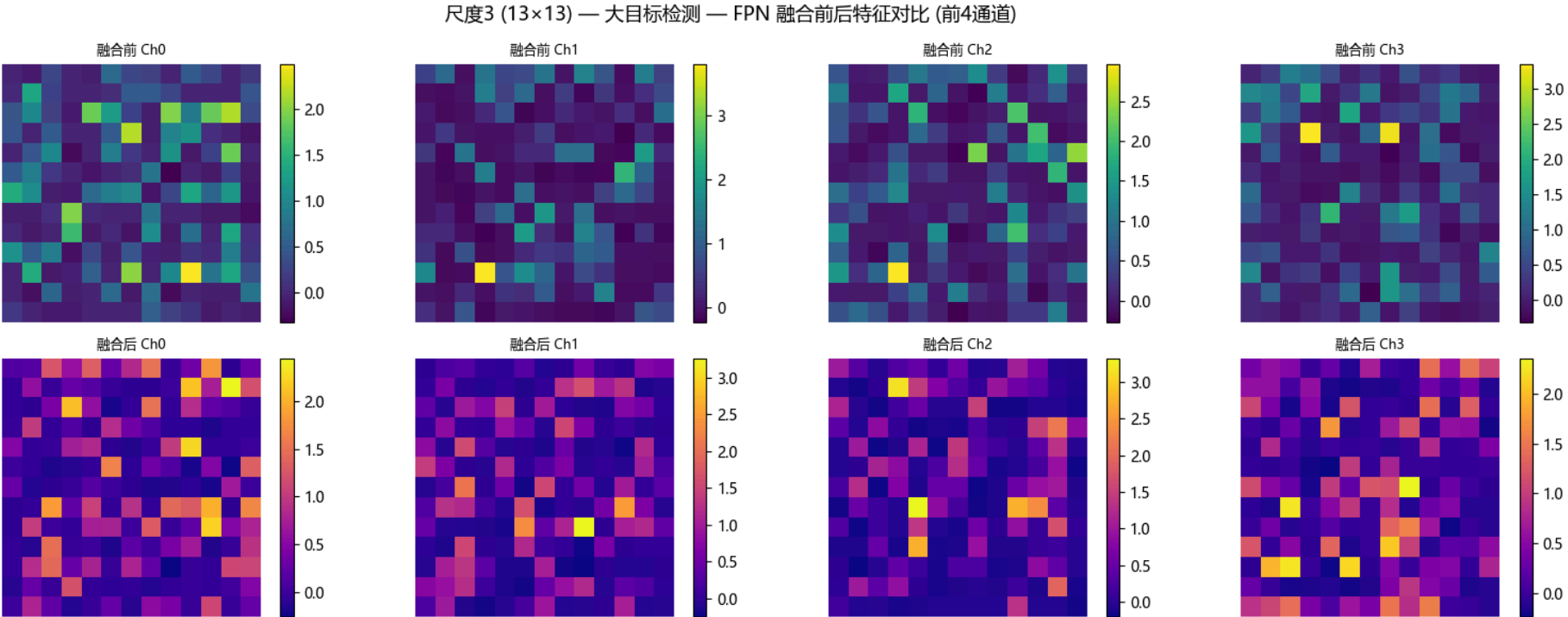
尺度3 (大目标): (1, 11, 13, 13)



总参数量: 2230.5K





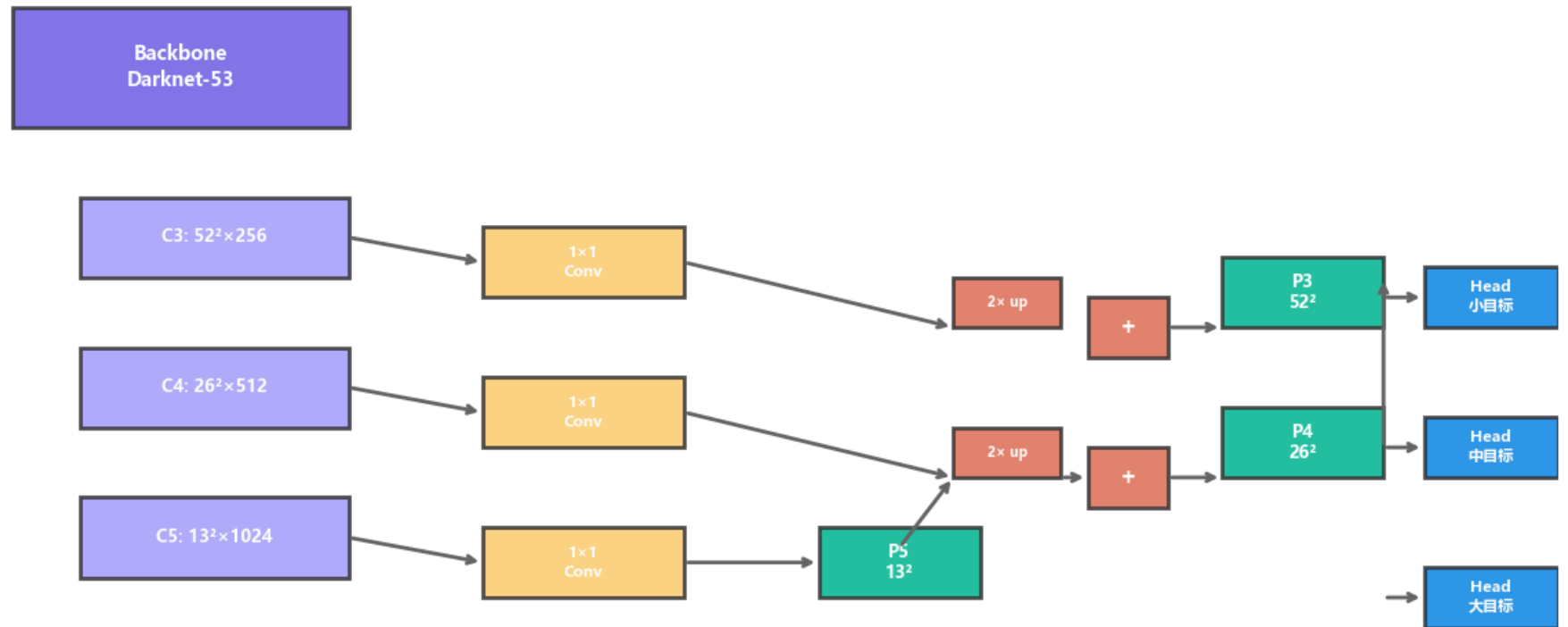


=====

FPN 结构图

=====

FPN (Feature Pyramid Network) — 完整信息流



=====

FPN 融合细节验证

=====

 P4 融合过程维度变化:

C5 lateral → P5: (1, 256, 13, 13)

P5 上采样 2×: (1, 256, 26, 26)

C4 lateral (1×1 Conv): (1, 256, 26, 26)

P4 = C4_lat + P5_up: (1, 256, 26, 26)  维度一致

P4 平滑后: (1, 256, 26, 26)

 FPN 参数量明细:

Lateral 1×1 Conv: 459.5K

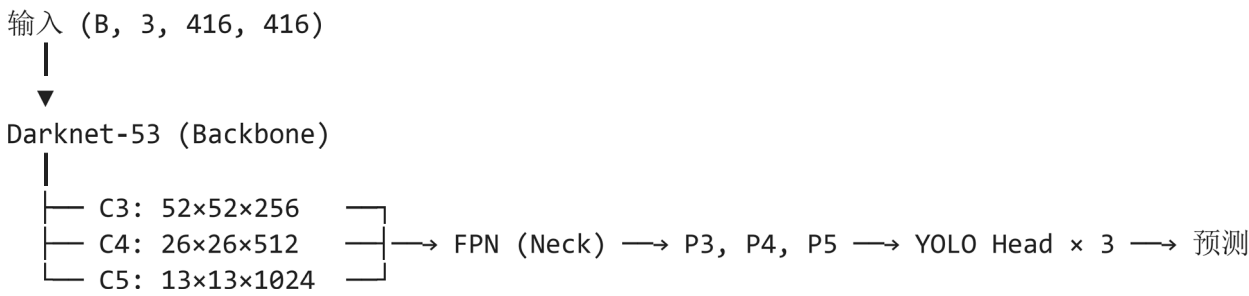
Smooth 3×3 Conv: 1769.5K

FPN 总参数量: 2229.0K

(仅为 Darknet-53 的 0.1%, 非常轻量!)

完整 YOLOv3 网络

将三个部件组合成完整的 YOLOv3:



YOLOv3 的关键设计

1. **Anchor Boxes (先验框)** : 每个尺度预设 3 个 anchor, 共 9 个 anchor

- 大尺度 (52×52): (10,13), (16,30), (33,23) — 检测小目标
- 中尺度 (26×26): (30,61), (62,45), (59,119) — 检测中目标

- 小尺度 (13×13): (116,90), (156,198), (373,326) — 检测大目标

2. 每个 Grid Cell 预测: $B \times (5 + C)$ 个值

- t_x, t_y : 相对于 anchor 的偏移
- t_w, t_h : 相对于 anchor 的缩放
- obj : 目标置信度
- C 个类别概率 (Sigmoid 多标签分类)

3. 坐标解码:

$$b_x = \sigma(t_x) + c_x \quad b_y = \sigma(t_y) + c_y$$

$$b_w = p_w \cdot e^{t_w} \quad b_h = p_h \cdot e^{t_h}$$

其中 c_x, c_y 是 Grid Cell 的左上角坐标, p_w, p_h 是 anchor 的宽高。

YOLOv3网络基础概念与架构设计

- 特征分层原理: 将网络特征分为浅层特征与深层网络特征, 深层特征维度更抽象、通道数更高。经过卷积层处理后, 1024维的输入会降维到512维, 输出尺寸为13×13的网络架构。
- 瓶颈结构作用: 使用1×1-3×3-1×1的卷积瓶颈结构, 先降通道减少计算量, 再做像素融合, 最后调整通道维度, 在保证特征提取能力的同时降低计算量。
- 多尺度输出设计: 针对不同尺寸目标设计三个输出尺度: 13×13对应大目标检测、26×26对应中等目标检测、52×52对应小目标检测; 检测难度随目标尺寸减小提升, 因此每一层都额外加深网络提升特征提取能力。
- 输出通道计算规则: 每个检测头的输出通道固定为 $3 \times (5 + \text{类别数})$, 其中5对应置信度、 c_x 、 c_y 、 w 、 h 五个基础参数, 3对应每个网格预设的3种不同形状锚框, 具体通道数随分类任务的类别数调整。

YOLOv3核心模块代码编写

- Convolutional set模块编写: 该模块输入输出图像尺寸不变, 仅调整通道数, 结构为两次降维升维卷积: 1×1卷积降通道、3×3卷积做像素融合、最后1×1卷积调整通道维度, 输入1024维输出512维可完成正常运算。
- 上采样模块实现: 直接调用PyTorch原生nn.Upsample实现两倍上采样, 使用临近插值法, 支持多种插值方法选择, 调用后13×13尺寸会变为26×26, 26×26会变为52×52。

- 整体网络拼接逻辑：从Darknet53主干网络获取三个初始输出，先处理13×13维度输出，经过卷积模块后，一部分直接输出大目标检测结果，另一部分做上采样后和主干网络26×26浅层输出做通道拼接；处理后一部分输出中等目标检测结果，另一部分继续上采样，和主干网络52×52浅层输出拼接，最后输出小目标检测结果，最终输出三个解耦的检测头。
- 面向对象代码编写规范：只对需要初始化的内容写在类的__init__方法中，不需要初始化的内容不要随意开辟内存，避免为了面向对象而面向对象，根据实际需求决定代码结构。

```
In [10]: import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from PIL import Image
import cv2
import os

# =====
# 1. 基础组件
# =====

class ConvBlock(nn.Module):
    """Darknet风格的卷积块：Conv2D + BatchNorm + LeakyReLU(0.1)"""
    def __init__(self, in_ch, out_ch, kernel_size=1, stride=1):
        super().__init__()
        padding = (kernel_size - 1) // 2
        self.conv = nn.Conv2d(in_ch, out_ch, kernel_size, stride, padding, bias=False)
        self.bn = nn.BatchNorm2d(out_ch)
        self.lrelu = nn.LeakyReLU(0.1, inplace=True)

    def forward(self, x):
        return self.lrelu(self.bn(self.conv(x)))

class ConvBlockNoBN(nn.Module):
    """不带BN和激活的卷积层（最后的预测层）"""
    def __init__(self, in_ch, out_ch, kernel_size=1, stride=1):
        super().__init__()
        padding = (kernel_size - 1) // 2
```

```

        self.conv = nn.Conv2d(in_ch, out_ch, kernel_size, stride, padding, bias=True)

    def forward(self, x):
        return self.conv(x)

class ResidualUnit(nn.Module):
    """
    Darknet 残差单元: 1x1降维 → 3x3升维 → +shortcut
    通道变化: ch → ch//2 → ch
    """
    def __init__(self, ch):
        super().__init__()
        self.block = nn.Sequential(
            ConvBlock(ch, ch // 2, 1, 1),
            ConvBlock(ch // 2, ch, 3, 1),
        )

    def forward(self, x):
        return x + self.block(x)

class ResidualBlock(nn.Module):
    """多个残差单元堆叠"""
    def __init__(self, ch, num_units):
        super().__init__()
        self.units = nn.Sequential(*[ResidualUnit(ch) for _ in range(num_units)])

    def forward(self, x):
        return self.units(x)

# =====
# 2. Darknet-53 骨干网络
# =====

class Darknet53(nn.Module):
    def __init__(self):
        super().__init__()
        self.stage0 = ConvBlock(3, 32, 3, 1)

```

```

self.stage1 = nn.Sequential(
    ConvBlock(32, 64, 3, 2),
    ResidualBlock(64, 1),
)

self.stage2 = nn.Sequential(
    ConvBlock(64, 128, 3, 2),
    ResidualBlock(128, 2),
)

self.stage3 = nn.Sequential(
    ConvBlock(128, 256, 3, 2),
    ResidualBlock(256, 8),
)

self.stage4 = nn.Sequential(
    ConvBlock(256, 512, 3, 2),
    ResidualBlock(512, 8),
)

self.stage5 = nn.Sequential(
    ConvBlock(512, 1024, 3, 2),
    ResidualBlock(1024, 4),
)

def forward(self, x):
    x = self.stage0(x)          # 416×416×32
    x = self.stage1(x)          # 208×208×64
    x = self.stage2(x)          # 104×104×128
    x = self.stage3(x)          # 52×52×256
    route_52 = x
    x = self.stage4(x)          # 26×26×512
    route_26 = x
    x = self.stage5(x)          # 13×13×1024
    route_13 = x
    return route_52, route_26, route_13

```

```

# =====
# 3. YOLOv3 检测头
# =====

```

```

class YOLOHead(nn.Module):
    """
    单个尺度的YOLOv3检测头
    输入特征图 → 5个ConvBlock (1×1和3×3交替) → 1个预测Conv (无BN)
    输出: [B, num_anchors×(5+num_classes), H, W]
    """
    def __init__(self, in_ch, num_classes=80, num_anchors=3):
        super().__init__()
        self.num_anchors = num_anchors
        self.num_classes = num_classes
        out_ch = num_anchors * (5 + num_classes)

        # 5层卷积: 1×1降维 → 3×3升维 → 1×1降维 → 3×3升维 → 1×1降维
        self.conv_block = nn.Sequential(
            ConvBlock(in_ch, in_ch // 2, 1, 1),
            ConvBlock(in_ch // 2, in_ch, 3, 1),
            ConvBlock(in_ch, in_ch // 2, 1, 1),
            ConvBlock(in_ch // 2, in_ch, 3, 1),
            ConvBlock(in_ch, in_ch // 2, 1, 1),
        )

        self.pred = ConvBlockNoBN(in_ch // 2, out_ch, 1, 1)

    def forward(self, x):
        x = self.conv_block(x)
        x = self.pred(x)
        return x # [B, 3×(5+C), H, W]

# =====
# 4. YOLOv3 完整网络
# =====

class YOLOv3(nn.Module):
    def __init__(self, num_classes=80, anchors_cfg='coco'):
        super().__init__()
        self.num_classes = num_classes

        # ----- Anchor 配置 -----
        if anchors_cfg == 'coco':

```

```

        self.anchors = torch.tensor([
            [10,13], [16,30], [33,23], # stride=8 (52×52) 大目标
            [30,61], [62,45], [59,119], # stride=16 (26×26) 中目标
            [116,90], [156,198], [373,326], # stride=32 (13×13) 小目标
        ], dtype=torch.float32)
    else:
        self.anchors = anchors_cfg

# ----- 骨干网络 -----
self.backbone = Darknet53()

# ----- 3 个检测头 -----
self.head_13 = YOLOHead(1024, num_classes, 3) # 13×13 (小目标)
self.head_26 = YOLOHead(512, num_classes, 3) # 26×26 (中目标)
self.head_52 = YOLOHead(256, num_classes, 3) # 52×52 (大目标)

# ----- FPN 上采样路径 -----
self.conv13_to_26 = nn.Sequential(
    ConvBlock(1024, 512, 1, 1),
    nn.Upsample(scale_factor=2, mode='nearest'),
)
self.conv26_reduce = ConvBlock(1024, 512, 1, 1)

self.conv26_to_52 = nn.Sequential(
    ConvBlock(512, 256, 1, 1),
    nn.Upsample(scale_factor=2, mode='nearest'),
)
self.conv52_reduce = ConvBlock(512, 256, 1, 1)

def forward(self, x):
    """
    输入: x [B, 3, 416, 416]
    返回: (out_13, out_26, out_52)
    """
    # 1) 骨干网络提取 3 个尺度特征
    route_52, route_26, route_13 = self.backbone(x)

    # 2) 13×13 分支 (小目标, stride=32)
    out_13 = self.head_13(route_13) # [B, 255, 13, 13]
    x_up_13 = self.conv13_to_26(route_13) # [B, 512, 26, 26]

```

```

# 3) 26×26 分支 (中目标, stride=16)
x_cat_26 = torch.cat([x_up_13, route_26], dim=1) # [B, 1024, 26, 26]
x_cat_26 = self.conv26_reduce(x_cat_26)         # [B, 512, 26, 26]
out_26 = self.head_26(x_cat_26)                 # [B, 255, 26, 26]
x_up_26 = self.conv26_to_52(x_cat_26)          # [B, 256, 52, 52]

# 4) 52×52 分支 (大目标, stride=8)
x_cat_52 = torch.cat([x_up_26, route_52], dim=1) # [B, 512, 52, 52]
x_cat_52 = self.conv52_reduce(x_cat_52)         # [B, 256, 52, 52]
out_52 = self.head_52(x_cat_52)                 # [B, 255, 52, 52]

return out_13, out_26, out_52

@property
def device(self):
    return next(self.parameters()).device

# =====
# 5. 解码: 网络原始输出 → 边界框坐标
# =====

def decode_scale(output, anchors, num_classes, stride, img_size=416):
    """
    解码单个尺度的 YOLO 输出

    参数:
        output: [B, 3×(5+C), H, W] – 网络原始输出
        anchors: [3, 2] – 该尺度的 3 个 anchor (w, h)
        num_classes: int
        stride: int – 下采样倍数
        img_size: int – 输入图像尺寸

    返回:
        boxes: [B, H×W×3, 6] – (x1, y1, x2, y2, conf, class_id)
    """
    B, _, H, W = output.shape
    num_anchors = len(anchors)

    # 重塑: [B, 3, 5+C, H, W] → [B, 3, H, W, 5+C]
    output = output.view(B, num_anchors, 5 + num_classes, H, W)

```

```
output = output.permute(0, 1, 3, 4, 2).contiguous()

# 分离各分量
tx = output[..., 0:1]
ty = output[..., 1:2]
tw = output[..., 2:3]
th = output[..., 3:4]
obj_conf = output[..., 4:5]
cls_logits = output[..., 5:]

# 创建网格坐标 (注意维度对齐)
grid_x = torch.arange(W, device=output.device).float().view(1, 1, 1, W, 1)
grid_y = torch.arange(H, device=output.device).float().view(1, 1, H, 1, 1)

# 解码
bx = torch.sigmoid(tx) + grid_x      # [B,3,H,W,1]
by = torch.sigmoid(ty) + grid_y
anchors = anchors.to(output.device).view(1, num_anchors, 1, 1, 2)
bw = anchors[..., 0:1] * torch.exp(tw)
bh = anchors[..., 1:2] * torch.exp(th)

# 置信度与类别
conf = torch.sigmoid(obj_conf)
cls_probs = torch.softmax(cls_logits, dim=-1)
cls_conf, cls_id = cls_probs.max(dim=-1, keepdim=True)
final_conf = conf * cls_conf

# 缩放至原图尺寸
bx *= stride
by *= stride
bw *= stride
bh *= stride

# 转 (x1, y1, x2, y2)
x1 = bx - bw / 2
y1 = by - bh / 2
x2 = bx + bw / 2
y2 = by + bh / 2

# 拼接 + 展平
boxes = torch.cat([x1, y1, x2, y2, final_conf, cls_id.float()], dim=-1)
```



```

boxes = boxes.view(B, -1, 6)
boxes[..., [0, 2]] = boxes[..., [0, 2]].clamp(0, img_size)
boxes[..., [1, 3]] = boxes[..., [1, 3]].clamp(0, img_size)

return boxes

def decode_all_scales(out_13, out_26, out_52, anchors, num_classes, img_size=416):
    """解码所有 3 个尺度的输出并合并"""
    strides = [32, 16, 8]
    scale_anchors = [
        anchors[6:9], # stride=32 (13x13, 小目标 → 大 anchor)
        anchors[3:6], # stride=16 (26x26)
        anchors[0:3], # stride=8 (52x52, 大目标 → 小 anchor)
    ]
    outputs = [out_13, out_26, out_52]

    all_boxes = []
    for output, anchor, stride in zip(outputs, scale_anchors, strides):
        boxes = decode_scale(output, anchor, num_classes, stride, img_size)
        all_boxes.append(boxes)

    return torch.cat(all_boxes, dim=1) # [B, total_boxes, 6]

# =====
# 6. NMS (Non-Maximum Suppression)
# =====

def nms(boxes, conf_thresh=0.5, iou_thresh=0.45):
    """
    非极大值抑制

    参数:
        boxes: [N, 6] - (x1,y1,x2,y2,conf,class_id)
        conf_thresh: 置信度阈值
        iou_thresh: IoU 阈值

    返回:
        keep: list[int] - 保留框的索引
    """

```

```
if boxes.shape[0] == 0:
    return []

# 按置信度过滤
mask = boxes[:, 4] > conf_thresh
boxes = boxes[mask]
if boxes.shape[0] == 0:
    return []

# 按置信度降序
order = boxes[:, 4].argsort(descending=True)
boxes = boxes[order]

keep_indices = []
while boxes.shape[0] > 0:
    # 选最高分框
    best = boxes[0:1]
    keep_indices.append(order[0].item())

    if boxes.shape[0] == 1:
        break

    rest = boxes[1:]

    # 计算 IoU
    x1 = torch.maximum(best[:, 0], rest[:, 0])
    y1 = torch.maximum(best[:, 1], rest[:, 1])
    x2 = torch.minimum(best[:, 2], rest[:, 2])
    y2 = torch.minimum(best[:, 3], rest[:, 3])
    inter = torch.clamp(x2 - x1, min=0) * torch.clamp(y2 - y1, min=0)

    area_best = (best[:, 2] - best[:, 0]) * (best[:, 3] - best[:, 1])
    area_rest = (rest[:, 2] - rest[:, 0]) * (rest[:, 3] - rest[:, 1])
    union = area_best + area_rest - inter
    iou = inter / (union + 1e-6)

    # 保留 IoU≤阈值 或 不同类别的框
    mask = (iou <= iou_thresh) | (best[:, 5] != rest[:, 5])
    boxes = rest[mask]
    order = order[1:][mask]
```

```
return keep_indices

def post_process(all_boxes, conf_thresh=0.5, iou_thresh=0.45):
    """
    对批量预测结果进行后处理

    参数:
        all_boxes: [B, N, 6]

    返回:
        results: list of [M, 6] - 每张图的检测结果
    """
    B = all_boxes.shape[0]
    results = []
    for i in range(B):
        boxes = all_boxes[i]

        # 置信度过滤
        mask = boxes[:, 4] > conf_thresh
        boxes = boxes[mask]
        if boxes.shape[0] == 0:
            results.append(torch.zeros((0, 6), device=all_boxes.device))
            continue

        # 按置信度排序
        order = boxes[:, 4].argsort(descending=True)
        boxes = boxes[order]

        # NMS 循环
        remaining = []
        while boxes.shape[0] > 0:
            remaining.append(boxes[0:1])
            if boxes.shape[0] == 1:
                break
            rest = boxes[1:]

            x1 = torch.maximum(boxes[0:1, 0], rest[:, 0])
            y1 = torch.maximum(boxes[0:1, 1], rest[:, 1])
            x2 = torch.minimum(boxes[0:1, 2], rest[:, 2])
            y2 = torch.minimum(boxes[0:1, 3], rest[:, 3])
```

```

        inter = torch.clamp(x2 - x1, min=0) * torch.clamp(y2 - y1, min=0)
        area1 = (boxes[0, 2] - boxes[0, 0]) * (boxes[0, 3] - boxes[0, 1])
        area2 = (rest[:, 2] - rest[:, 0]) * (rest[:, 3] - rest[:, 1])
        iou = inter / (area1 + area2 - inter + 1e-6)
        mask = (iou <= iou_thresh) | (boxes[0, 5] != rest[:, 5])
        boxes = rest[mask]

    results.append(torch.cat(remaining, dim=0) if remaining
                    else torch.zeros((0, 6), device=all_boxes.device))
    return results

# =====
# 7. 加载 Darknet 预训练权重
# =====

def load_darknet_weights(model, weights_path):
    """
    加载 Darknet 预训练权重到 PyTorch YOLOv3 模型

    Darknet 权重格式:
    [int32x5: header] → [float32...: 权重数据]
    每层顺序 (有 BN): bn_bias, bn_weight, bn_mean, bn_var, conv_weight
    每层顺序 (无 BN): conv_bias, conv_weight
    """
    with open(weights_path, 'rb') as f:
        header = np.fromfile(f, dtype=np.int32, count=5)
        print(f'[加载权重] header: {header}')
        weights = np.fromfile(f, dtype=np.float32)
        print(f'[加载权重] 总权重数: {len(weights)}')

    # 收集所有 Conv2d 层, 区分是否带 BN
    conv_with_bn = []
    conv_without_bn = []

    def gather(module):
        for name, child in module.named_children():
            if isinstance(child, nn.Conv2d):
                # 检查下一个兄弟节点是否是 BN
                parent = module
                children_list = list(parent.named_children())

```

```
        idx = [i for i, (n, _) in enumerate(children_list) if n == name][0]
        if idx + 1 < len(children_list) and \
            isinstance(children_list[idx + 1][1], nn.BatchNorm2d):
            conv_with_bn.append(child)
        else:
            conv_without_bn.append(child)
    else:
        gather(child)
gather(model)

print(f'[加载权重] 含BN: {len(conv_with_bn)}, 无BN: {len(conv_without_bn)}')

# 收集所有 BN 层 (按遍历顺序)
bn_layers = []
for m in model.modules():
    if isinstance(m, nn.BatchNorm2d):
        bn_layers.append(m)

ptr = 0
# 加载带 BN 的层: bn_bias, bn_weight, bn_mean, bn_var, conv_weight
for i, conv in enumerate(conv_with_bn):
    n = conv.out_channels
    bn_layers[i].bias.data.copy_(torch.from_numpy(weights[ptr:ptr + n]))
    ptr += n
    bn_layers[i].weight.data.copy_(torch.from_numpy(weights[ptr:ptr + n]))
    ptr += n
    bn_layers[i].running_mean.data.copy_(torch.from_numpy(weights[ptr:ptr + n]))
    ptr += n
    bn_layers[i].running_var.data.copy_(torch.from_numpy(weights[ptr:ptr + n]))
    ptr += n

    num_w = conv.weight.numel()
    conv.weight.data.copy_(torch.from_numpy(weights[ptr:ptr + num_w]).view_as(conv.weight))
    ptr += num_w

# 加载不带 BN 的层: conv_bias, conv_weight
for conv in conv_without_bn:
    n = conv.out_channels
    conv.bias.data.copy_(torch.from_numpy(weights[ptr:ptr + n]))
    ptr += n
    num_w = conv.weight.numel()
```

```

conv.weight.data.copy_(torch.from_numpy(weights[ptr:ptr + num_w]).view_as(conv.weight))
ptr += num_w

print(f'[加载权重] 完成! 使用 {ptr}/{len(weights)} 个权重')
if ptr != len(weights):
    print(f'[加载权重] ⚠️ 有 {len(weights)-ptr} 个未加载 (模型结构与原版差异)')

if __name__ == '__main__':
    test_forward()

```

```

=====
YOLOv3 前向传播测试
=====

```

总参数量: 56,591,197

out_13: torch.Size([1, 255, 13, 13]) (小目标, stride=32)

out_26: torch.Size([1, 255, 26, 26]) (中目标, stride=16)

out_52: torch.Size([1, 255, 52, 52]) (大目标, stride=8)

解码后总框数: 10647

理论值: $13 \times 13 \times 3 + 26 \times 26 \times 3 + 52 \times 52 \times 3 = 10647$

NMS 后: 0 个框

✅ 前向传播测试通过!

可视化 YOLO 输出的 9 个通道图

每个尺度的输出有 255 个通道: 3 个 anchor × (tx, ty, tw, th, **obj_conf**, 80 个类别)。

这里提取每个 anchor 的 **obj_conf** 通道 (目标置信度), 共 3 尺度 × 3 anchor = **9 张热图**, 直观展示网络"看"到了什么。

```

In [9]: # =====
# 可视化 YOLO 输出: 3 尺度 × 3 anchor = 9 张 obj_conf 热图
# 使用随机输入, 不依赖图片文件
# =====

from net_full import YOLOv3

```

```
def visualize_yolo_channels():  
    """  
    用随机噪声作为输入，提取 YOLOv3 三个输出尺度中  
    每个 anchor 的 obj_conf 通道，绘制 3×3 的热图网格。  
    """  
  
    model = YOLOv3(num_classes=80)  
    model.eval()  
  
    # 随机输入（代替真实图片）  
    x = torch.randn(1, 3, 416, 416)  
    print(f'使用随机噪声输入: {x.shape}')  
    with torch.no_grad():  
        out_13, out_26, out_52 = model(x)  
  
    scales = [  
        ('13×13 (stride=32)', out_13),  
        ('26×26 (stride=16)', out_26),  
        ('52×52 (stride=8)', out_52),  
    ]  
  
    fig, axes = plt.subplots(3, 3, figsize=(12, 10))  
    anchor_names = ['Anchor 1 (小)', 'Anchor 2 (中)', 'Anchor 3 (大)']  
  
    for row, (scale_name, out) in enumerate(scales):  
        B, C, H, W = out.shape  
        out_reshaped = out.view(B, 3, 85, H, W)  
        obj_confs = torch.sigmoid(out_reshaped[:, :, 4, :, :])  
  
        for col in range(3):  
            ax = axes[row, col]  
            heatmap = obj_confs[0, col].cpu().numpy()  
  
            im = ax.imshow(heatmap, cmap='hot', interpolation='nearest',  
                           vmin=0, vmax=1)  
            ax.set_title(f'{scale_name}\n{anchor_names[col]}', fontsize=9)  
            ax.axis('off')  
  
            if H <= 13:  
                for i in range(H):  
                    for j in range(W):
```

```
        val = heatmap[i, j]
        ax.text(j, i, f'{val:.2f}', ha='center', va='center',
                fontsize=6, color='white' if val < 0.7 else 'black')

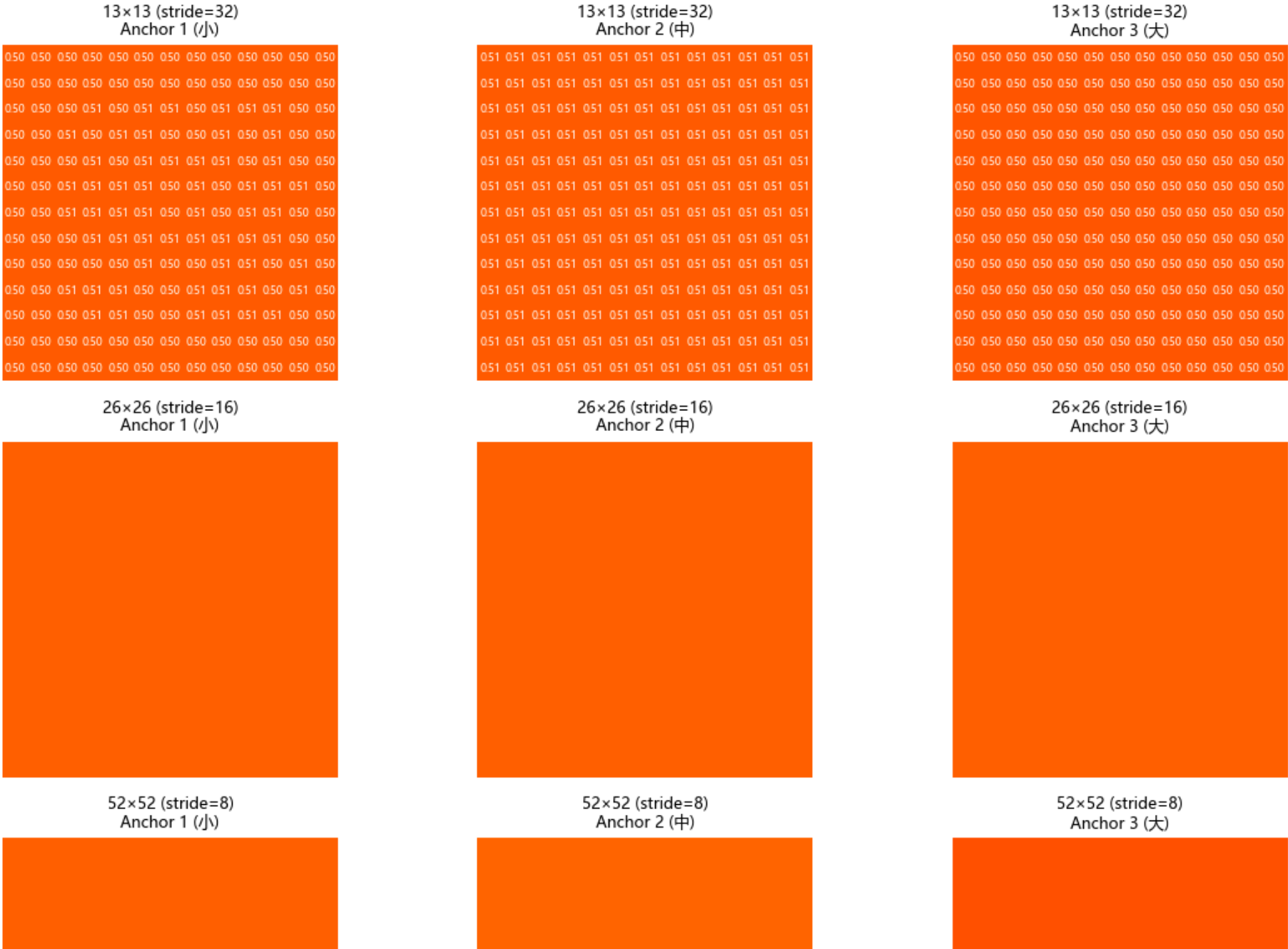
plt.suptitle('YOLOv3 obj_conf 热图 (随机噪声输入)', fontsize=14)
plt.tight_layout()
plt.show()

# 统计
print(f'\n📊 各尺度 obj_conf 统计:')
for scale_name, out in scales:
    B, C, H, W = out.shape
    out_r = out.view(B, 3, 85, H, W)
    confs = torch.sigmoid(out_r[:, :, 4, :, :])
    print(f' {scale_name}: '
          f'min={confs.min().item():.4f}, '
          f'max={confs.max().item():.4f}, '
          f'mean={confs.mean().item():.4f}, '
          f'>0.5 = {(confs > 0.5).sum().item()} 个格子')

visualize_yolo_channels()
```

使用随机噪声输入: torch.Size([1, 3, 416, 416])

YOLOv3 obj_conf 热图 (随机噪声输入)





各尺度 obj_conf 统计:

13×13 (stride=32): min=0.4976, max=0.5073, mean=0.5033, >0.5 = 338 个格子
 26×26 (stride=16): min=0.5132, max=0.5148, mean=0.5141, >0.5 = 2028 个格子
 52×52 (stride=8): min=0.4907, max=0.5170, mean=0.5074, >0.5 = 5408 个格子

```
In [ ]: from torch.utils.data import Dataset
import math

class YOLODataset(Dataset):
    def __init__(self, root_path, img_size=416, num_classes=80,
                 anchors=None, anchor_groups=None):
        super().__init__()
        self.img_size = img_size
        self.num_classes = num_classes
        self.dataset = open(root_path, 'r', encoding='utf-8').readlines()

        # 默认 COCO anchor
        if anchors is None:
            self.anchors = torch.tensor([
                [10,13],[16,30],[33,23],
                [30,61],[62,45],[59,119],
                [116,90],[156,198],[373,326]], dtype=torch.float32)
        else:
            self.anchors = anchors

        # 每个尺度对应的 anchor 分组
        if anchor_groups is None:
            self.anchor_groups = {
                13: self.anchors[6:9],    # 大 anchor 检测小目标
                26: self.anchors[3:6],    # 中 anchor 检测中目标
                52: self.anchors[0:3],    # 小 anchor 检测大目标
```

```

    }
else:
    self.anchor_groups = anchor_groups

    print(f'[YOLODataset] 加载 {len(self.dataset)} 张图片, '
          f'{num_classes} 类')

def __len__(self):
    return len(self.dataset)

def __getitem__(self, idx):
    line = self.dataset[idx].strip()
    parts = line.split()
    img_path = parts[0]

    # — 读图 —
    img = Image.open(img_path).convert('RGB')
    img = img.resize((self.img_size, self.img_size))
    img_tensor = torch.from_numpy(np.array(img)).float().permute(2,0,1) / 255.0

    # — 解析标注框 —
    # parts[1:] 是 flat 列表: [cls, cx, cy, w, h, cls, ...]
    nums = np.array([float(x) for x in parts[1:]], dtype=np.float32)
    boxes = nums.reshape(-1, 5) # [N, 5]

    # — 初始化 label 张量 —
    # 每个尺度: [feat_size, feat_size, 3, 5+num_classes]
    labels = {}
    for feat_size in [13, 26, 52]:
        labels[feat_size] = torch.zeros(
            feat_size, feat_size, 3, 5 + self.num_classes)

    # — 编码每个标注框 —
    for box in boxes:
        cls_id, cx, cy, bw, bh = box

        for feat_size, anchors_fs in self.anchor_groups.items():
            stride = self.img_size // feat_size # 32, 16, 8

            # 计算网格索引和格内偏移
            cx_idx = cx / stride

```

```

        cy_idx = cy / stride
        grid_x = int(math.floor(cx_idx))
        grid_y = int(math.floor(cy_idx))
        offset_x = cx_idx - grid_x
        offset_y = cy_idx - grid_y

        # 超出特征图范围则跳过
        if grid_x < 0 or grid_x >= feat_size: continue
        if grid_y < 0 or grid_y >= feat_size: continue

        # 遍历该尺度的 3 个 anchor
        for a_idx in range(3):
            anchor_w = anchors_fs[a_idx, 0]
            anchor_h = anchors_fs[a_idx, 1]

            # 编码: offset_w = log(gt_w / anchor_w)
            offset_w = float(np.log(bw / anchor_w))
            offset_h = float(np.log(bh / anchor_h))

            # one-hot 类别
            cls_onehot = torch.zeros(self.num_classes)
            cls_onehot[int(cls_id)] = 1.0

            # 填入 label 张量
            val = [1.0, offset_x, offset_y, offset_w, offset_h]
            labels[feat_size][grid_y, grid_x, a_idx] = \
                torch.tensor(val + cls_onehot.tolist(), dtype=torch.float32)

    return labels[13], labels[26], labels[52], img_tensor

# =====
# Dataset 测试
# =====

def test_dataset():
    """生成模拟数据, 测试 Dataset 是否能正常运行"""
    import tempfile, shutil

    # 创建临时目录
    tmpdir = tempfile.mkdtemp()

```

```
img_dir = os.path.join(tmpdir, 'images')
os.makedirs(img_dir)

# 生成 2 张模拟图片和 train.txt
train_lines = []
for i in range(2):
    img = np.random.randint(0, 256, (300, 500, 3), dtype=np.uint8)
    img_path = os.path.join(img_dir, f'{i}.jpg').replace('\\', '/')
    cv2.imwrite(img_path, img)

    # 每张图随机 1~3 个目标
    num_obj = np.random.randint(1, 4)
    line = img_path
    for _ in range(num_obj):
        xc = np.random.uniform(0.1, 0.9)
        yc = np.random.uniform(0.1, 0.9)
        w = np.random.uniform(0.05, 0.3)
        h = np.random.uniform(0.05, 0.3)
        cls = np.random.randint(0, 3)
        line += f' {cls} {xc:.4f} {yc:.4f} {w:.4f} {h:.4f}'
    train_lines.append(line)

train_txt = os.path.join(tmpdir, 'train.txt')
with open(train_txt, 'w', encoding='utf-8') as f:
    f.write('\n'.join(train_lines))

# 测试 Dataset
ds = YOLODataset(train_txt, num_classes=3)
print(f'\n📁 Dataset 测试:')
print(f'  样本数: {len(ds)}')

l13, l26, l52, img = ds[0]
print(f'  图片张量: {img.shape}')
print(f'  label_13: {l13.shape} (有目标格子: {(l13[...,0]==1).sum()})')
print(f'  label_26: {l26.shape} (有目标格子: {(l26[...,0]==1).sum()})')
print(f'  label_52: {l52.shape} (有目标格子: {(l52[...,0]==1).sum()})')

# 验证编码: 有 conf=1 的位置, offset 应该非零
for feat_size, lbl in [(13, l13), (26, l26), (52, l52)]:
    mask = lbl[..., 0] == 1
    if mask.any():
        # 验证逻辑 (根据上下文推断)
```

```
offsets = lbl[..., 1:5][mask]
print(f'    {feat_size}×{feat_size}: {mask.sum()} 个正样本, '
      f'offset 范围: [{offsets.min():.2f}, {offsets.max():.2f}]')

print('✅ Dataset 测试通过!')
shutil.rmtree(tmpdir)
```